HTML FUNDAMENTALS AND SEMANTIC ELEMENTS

A Practical Educational Book for BSIT Students

Six chapters | 30 illustrated lesson pages | HTML examples and explanations

Table of Contents

Chapter 1: Introduction to HTML

1.1 What Is HTML?

1.2 History and Evolution of HTML

1.3 Understanding Modern HTML

1.4 Basic HTML Document Structure

1.5 Tags, Elements, and Attributes

Chapter 2: Basic HTML Elements

2.1 Headings and Paragraphs

2.2 Text Formatting and Meaning

2.3 Lists and Hyperlinks

2.4 Images and Multimedia

2.5 Tables for Structured Data

Chapter 3: HTML Document Structure

3.1 DOCTYPE, HTML, Head, and Body

3.2 Metadata and Page Titles

3.3 Block and Inline Flow

3.4 Div and Span Elements

3.5 Nesting and Valid Markup

Chapter 4: HTML Semantic Elements

4.1 Introduction to Semantic HTML

4.2 Header and Navigation

4.3 Main and Section Elements

4.4 Article and Aside Elements

4.5 Footer, Figure, and Figcaption

Chapter 5: Creating Webpage Layouts

5.1 Organizing Page Content

5.2 Building a Semantic Webpage

5.3 Forms and Input Elements

5.4 Accessibility and Best Practices

5.5 Common HTML Mistakes

Chapter 6: Practical HTML Examples

6.1 Creating a Basic Webpage

6.2 Building a Student Profile Page

6.3 Creating a Navigation Menu

6.4 Designing an Educational Website

6.5 Complete Semantic HTML Page

Chapter 1: Introduction to HTML

1.1 What Is HTML?

HTML stands for HyperText Markup Language. It describes the meaning and organization of content on a webpage. A browser reads HTML and constructs a document structure that can be styled with CSS and made interactive with JavaScript.

HTML example

```
<h1>Welcome to My Website</h1>
<p>This is my first paragraph.</p>
```

Understanding the example

Read the code from top to bottom. Notice how each opening tag identifies a piece of content, how attributes add information, and how nested elements establish relationships. The browser interprets this markup as a structured document rather than simply displaying the tag characters.

When applying this pattern in a real website, replace the sample words and file names with your own content. Save the document as an HTML file and open it in a modern browser to inspect the result. For images, store the image file separately so it can be replaced without rewriting the lesson.

Practical design note

Use the most meaningful HTML element for the job. Avoid selecting tags only because of their default appearance. Add CSS for layout and color, and check that the content remains understandable when styling is disabled. This separation makes a project easier to revise and maintain.

1.2 History and Evolution of HTML

HTML originated with the early World Wide Web. Over time, browser implementations and standards introduced improved document structure, forms, multimedia, and accessibility. Modern HTML is maintained as a continuously evolving Living Standard.

HTML example

```
<!doctype html>
<html lang="en">
  <head><title>Modern HTML</title></head>
  <body><h1>Hello</h1></body>
</html>
```

Understanding the example

Read the code from top to bottom. Notice how each opening tag identifies a piece of content, how attributes add information, and how nested elements establish relationships. The browser interprets this markup as a structured document rather than simply displaying the tag characters.

When applying this pattern in a real website, replace the sample words and file names with your own content. Save the document as an HTML file and open it in a modern browser to inspect the result. For images, store the image file separately so it can be replaced without rewriting the lesson.

Practical design note

Use the most meaningful HTML element for the job. Avoid selecting tags only because of their default appearance. Add CSS for layout and color, and check that the content remains understandable when styling is disabled. This separation makes a project easier to revise and maintain.

1.3 Understanding Modern HTML

Modern HTML includes native video and audio elements, form controls, and semantic landmarks. HTML defines structure; CSS defines presentation; JavaScript handles behavior. Keeping these responsibilities separate makes sites easier to maintain.

HTML example

```
<video controls width="320">
  <source src="lesson.mp4" type="video/mp4">
</video>
```

Understanding the example

Read the code from top to bottom. Notice how each opening tag identifies a piece of content, how attributes add information, and how nested elements establish relationships. The browser interprets this markup as a structured document rather than simply displaying the tag characters.

When applying this pattern in a real website, replace the sample words and file names with your own content. Save the document as an HTML file and open it in a modern browser to inspect the result. For images, store the image file separately so it can be replaced without rewriting the lesson.

Practical design note

Use the most meaningful HTML element for the job. Avoid selecting tags only because of their default appearance. Add CSS for layout and color, and check that the content remains understandable when styling is disabled. This separation makes a project easier to revise and maintain.

1.4 Basic HTML Document Structure

A complete document begins with a doctype declaration, followed by the html root element. The head contains metadata, while the body contains visible content. The lang attribute identifies the document language.

HTML example

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Student Page</title>
</head>
<body><main><h1>Student Page</h1></main></body>
</html>
```

Understanding the example

Read the code from top to bottom. Notice how each opening tag identifies a piece of content, how attributes add information, and how nested elements establish relationships. The browser interprets this markup as a structured document rather than simply displaying the tag characters.

When applying this pattern in a real website, replace the sample words and file names with your own content. Save the document as an HTML file and open it in a modern browser to inspect the result. For images, store the image file separately so it can be replaced without rewriting the lesson.

Practical design note

Use the most meaningful HTML element for the job. Avoid selecting tags only because of their default appearance. Add CSS for layout and color, and check that the content remains understandable when styling is disabled. This separation makes a project easier to revise and maintain.

1.5 Tags, Elements, and Attributes

An HTML tag marks the beginning or end of an element. An element may contain text or other elements. Attributes supply additional information, such as a link destination or image description. Not all elements have closing tags: img and br are void elements.

HTML example

```
<a href="https://example.org">Visit example</a>
<img src="campus.jpg" alt="Campus entrance">
```

Understanding the example

Read the code from top to bottom. Notice how each opening tag identifies a piece of content, how attributes add information, and how nested elements establish relationships. The browser interprets this markup as a structured document rather than simply displaying the tag characters.

When applying this pattern in a real website, replace the sample words and file names with your own content. Save the document as an HTML file and open it in a modern browser to inspect the result. For images, store the image file separately so it can be replaced without rewriting the lesson.

Practical design note

Use the most meaningful HTML element for the job. Avoid selecting tags only because of their default appearance. Add CSS for layout and color, and check that the content remains understandable when styling is disabled. This separation makes a project easier to revise and maintain.

Chapter 1 takeaway: The concepts in this chapter provide building blocks for readable, maintainable and accessible web documents.

Chapter 2: Basic HTML Elements

2.1 Headings and Paragraphs

Heading elements h1 through h6 establish a meaningful hierarchy. A page usually has a main h1 describing its subject. Paragraphs group related sentences. Do not choose heading levels only for visual size; use CSS for appearance.

HTML example

```
<h1>Web Development</h1>
<h2>Introduction</h2>
<p>HTML provides structure.</p>
```

Understanding the example

Read the code from top to bottom. Notice how each opening tag identifies a piece of content, how attributes add information, and how nested elements establish relationships. The browser interprets this markup as a structured document rather than simply displaying the tag characters.

When applying this pattern in a real website, replace the sample words and file names with your own content. Save the document as an HTML file and open it in a modern browser to inspect the result. For images, store the image file separately so it can be replaced without rewriting the lesson.

Practical design note

Use the most meaningful HTML element for the job. Avoid selecting tags only because of their default appearance. Add CSS for layout and color, and check that the content remains understandable when styling is disabled. This separation makes a project easier to revise and maintain.

2.2 Text Formatting and Meaning

HTML provides elements that express meaning. Strong indicates importance and em indicates emphasis. The code element identifies computer code. The mark element highlights relevant text. These elements are more informative than styling text with generic spans alone.

HTML example

```
<p><strong>Important:</strong> Save your work.</p>
<p>Press <kbd>Ctrl</kbd> + <kbd>S</kbd>.</p>
<p>Use the <code>main</code> element.</p>
```

Understanding the example

Read the code from top to bottom. Notice how each opening tag identifies a piece of content, how attributes add information, and how nested elements establish relationships. The browser interprets this markup as a structured document rather than simply displaying the tag characters.

When applying this pattern in a real website, replace the sample words and file names with your own content. Save the document as an HTML file and open it in a modern browser to inspect the result. For images, store the image file separately so it can be replaced without rewriting the lesson.

Practical design note

Use the most meaningful HTML element for the job. Avoid selecting tags only because of their default appearance. Add CSS for layout and color, and check that the content remains understandable when styling is disabled. This separation makes a project easier to revise and maintain.

2.3 Lists and Hyperlinks

Unordered lists use ul and li; ordered lists use ol and li. Anchor elements create links using href. Link text should explain the destination rather than say only click here. Relative links point to pages within the same site.

HTML example

```
<nav aria-label="Lessons">
  <ul>
    <li><a href="intro.html">Introduction</a></li>
    <li><a href="topics.html">Topics</a></li>
  </ul>
</nav>
```

Understanding the example

Read the code from top to bottom. Notice how each opening tag identifies a piece of content, how attributes add information, and how nested elements establish relationships. The browser interprets this markup as a structured document rather than simply displaying the tag characters.

When applying this pattern in a real website, replace the sample words and file names with your own content. Save the document as an HTML file and open it in a modern browser to inspect the result. For images, store the image file separately so it can be replaced without rewriting the lesson.

Practical design note

Use the most meaningful HTML element for the job. Avoid selecting tags only because of their default appearance. Add CSS for layout and color, and check that the content remains understandable when styling is disabled. This separation makes a project easier to revise and maintain.

2.4 Images and Multimedia

The img element embeds an image and requires useful alternative text unless it is purely decorative. The figure and figcaption elements associate an illustration with a caption. Audio and video elements can expose native playback controls.

HTML example

```
<figure>
  <img src="network.png" alt="Three computers connected to a router">
  <figcaption>Figure 2.1: Simple network.</figcaption>
</figure>
```

Understanding the example

Read the code from top to bottom. Notice how each opening tag identifies a piece of content, how attributes add information, and how nested elements establish relationships. The browser interprets this markup as a structured document rather than simply displaying the tag characters.

When applying this pattern in a real website, replace the sample words and file names with your own content. Save the document as an HTML file and open it in a modern browser to inspect the result. For images, store the image file separately so it can be replaced without rewriting the lesson.

Practical design note

Use the most meaningful HTML element for the job. Avoid selecting tags only because of their default appearance. Add CSS for layout and color, and check that the content remains understandable when styling is disabled. This separation makes a project easier to revise and maintain.

2.5 Tables for Structured Data

Tables organize data with meaningful rows and columns. Use caption for a title, th for header cells, and scope to identify the relevant row or column. Tables should not be used to create the overall page layout.

HTML example

```
<table>
  <caption>Course Schedule</caption>
  <tr><th scope="col">Day</th><th scope="col">Topic</th></tr>
  <tr><td>Monday</td><td>HTML</td></tr>
</table>
```

Understanding the example

Read the code from top to bottom. Notice how each opening tag identifies a piece of content, how attributes add information, and how nested elements establish relationships. The browser interprets this markup as a structured document rather than simply displaying the tag characters.

When applying this pattern in a real website, replace the sample words and file names with your own content. Save the document as an HTML file and open it in a modern browser to inspect the result. For images, store the image file separately so it can be replaced without rewriting the lesson.

Practical design note

Use the most meaningful HTML element for the job. Avoid selecting tags only because of their default appearance. Add CSS for layout and color, and check that the content remains understandable when styling is disabled. This separation makes a project easier to revise and maintain.

Chapter 2 takeaway: The concepts in this chapter provide building blocks for readable, maintainable and accessible web documents.

Chapter 3: HTML Document Structure

3.1 DOCTYPE, HTML, Head, and Body

The doctype signals standards-mode HTML parsing. The html element encloses the document; head stores metadata and linked resources; body contains content rendered for users. A logical document structure benefits both people and tools.

HTML example

```
<!doctype html>
<html lang="en">
<head><title>Library</title></head>
<body><h1>Digital Library</h1></body>
</html>
```

Understanding the example

Read the code from top to bottom. Notice how each opening tag identifies a piece of content, how attributes add information, and how nested elements establish relationships. The browser interprets this markup as a structured document rather than simply displaying the tag characters.

When applying this pattern in a real website, replace the sample words and file names with your own content. Save the document as an HTML file and open it in a modern browser to inspect the result. For images, store the image file separately so it can be replaced without rewriting the lesson.

Practical design note

Use the most meaningful HTML element for the job. Avoid selecting tags only because of their default appearance. Add CSS for layout and color, and check that the content remains understandable when styling is disabled. This separation makes a project easier to revise and maintain.

3.2 Metadata and Page Titles

Metadata helps browsers interpret the page. The charset declaration specifies character encoding. The viewport meta tag helps pages adapt to mobile devices. A descriptive title appears in tabs, bookmarks, and many search results.

HTML example

```
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <meta name="description" content="BSIT study materials">
  <title>BSIT Digital Library</title>
</head>
```

Understanding the example

Read the code from top to bottom. Notice how each opening tag identifies a piece of content, how attributes add information, and how nested elements establish relationships. The browser interprets this markup as a structured document rather than simply displaying the tag characters.

When applying this pattern in a real website, replace the sample words and file names with your own content. Save the document as an HTML file and open it in a modern browser to inspect the result. For images, store the image file separately so it can be replaced without rewriting the lesson.

Practical design note

Use the most meaningful HTML element for the job. Avoid selecting tags only because of their default appearance. Add CSS for layout and color, and check that the content remains understandable when styling is disabled. This separation makes a project easier to revise and maintain.

3.3 Block and Inline Flow

Some HTML elements commonly form block-level boxes, such as paragraphs and sections, while others flow within text, such as links and emphasis. CSS can change layout behavior, so the semantic meaning of the element should guide the choice.

HTML example

```
<section>
  <h2>News</h2>
  <p>Read the <a href="update.html">latest update</a>.</p>
</section>
```

Understanding the example

Read the code from top to bottom. Notice how each opening tag identifies a piece of content, how attributes add information, and how nested elements establish relationships. The browser interprets this markup as a structured document rather than simply displaying the tag characters.

When applying this pattern in a real website, replace the sample words and file names with your own content. Save the document as an HTML file and open it in a modern browser to inspect the result. For images, store the image file separately so it can be replaced without rewriting the lesson.

Practical design note

Use the most meaningful HTML element for the job. Avoid selecting tags only because of their default appearance. Add CSS for layout and color, and check that the content remains understandable when styling is disabled. This separation makes a project easier to revise and maintain.

3.4 Div and Span Elements

The div and span elements are generic containers without built-in semantic meaning. Use them for grouping or styling when no more specific element applies. For navigation, articles, and page landmarks, prefer the corresponding semantic elements.

HTML example

```
<div class="card">
  <h2>Book Details</h2>
  <p>Level: <span class="level">First Year</span></p>
</div>
```

Understanding the example

Read the code from top to bottom. Notice how each opening tag identifies a piece of content, how attributes add information, and how nested elements establish relationships. The browser interprets this markup as a structured document rather than simply displaying the tag characters.

When applying this pattern in a real website, replace the sample words and file names with your own content. Save the document as an HTML file and open it in a modern browser to inspect the result. For images, store the image file separately so it can be replaced without rewriting the lesson.

Practical design note

Use the most meaningful HTML element for the job. Avoid selecting tags only because of their default appearance. Add CSS for layout and color, and check that the content remains understandable when styling is disabled. This separation makes a project easier to revise and maintain.

3.5 Nesting and Valid Markup

HTML elements must be nested correctly. A child element should close before its parent closes. Some elements have content restrictions: a paragraph cannot contain a section, and interactive controls should not be improperly nested. Validate pages regularly.

HTML example

```
<!-- Correct nesting -->
<p>Read <strong>carefully</strong>.</p>
<!-- Avoid overlapping closing tags -->
```

Understanding the example

Read the code from top to bottom. Notice how each opening tag identifies a piece of content, how attributes add information, and how nested elements establish relationships. The browser interprets this markup as a structured document rather than simply displaying the tag characters.

When applying this pattern in a real website, replace the sample words and file names with your own content. Save the document as an HTML file and open it in a modern browser to inspect the result. For images, store the image file separately so it can be replaced without rewriting the lesson.

Practical design note

Use the most meaningful HTML element for the job. Avoid selecting tags only because of their default appearance. Add CSS for layout and color, and check that the content remains understandable when styling is disabled. This separation makes a project easier to revise and maintain.

Chapter 3 takeaway: The concepts in this chapter provide building blocks for readable, maintainable and accessible web documents.

Chapter 4: HTML Semantic Elements

4.1 Introduction to Semantic HTML

Semantic HTML uses elements whose names describe their purpose. Examples include header, nav, main, article, aside, and footer. Meaningful structure supports accessibility, maintainability, and machine interpretation of documents.

HTML example

```
<header><h1>Campus News</h1></header>
<main><article><h2>Announcement</h2></article></main>
<footer><p>School Portal</p></footer>
```

Understanding the example

Read the code from top to bottom. Notice how each opening tag identifies a piece of content, how attributes add information, and how nested elements establish relationships. The browser interprets this markup as a structured document rather than simply displaying the tag characters.

When applying this pattern in a real website, replace the sample words and file names with your own content. Save the document as an HTML file and open it in a modern browser to inspect the result. For images, store the image file separately so it can be replaced without rewriting the lesson.

Practical design note

Use the most meaningful HTML element for the job. Avoid selecting tags only because of their default appearance. Add CSS for layout and color, and check that the content remains understandable when styling is disabled. This separation makes a project easier to revise and maintain.

4.2 Header and Navigation

The header element introduces a page or section. The nav element groups major navigation links. A page can have more than one header or nav when they serve different regions. Descriptive navigation labels help visitors orient themselves.

HTML example

```
<header>
  <h1>BSIT E-Library</h1>
  <nav aria-label="Primary">
    <a href="index.html">Home</a>
    <a href="library.html">Library</a>
  </nav>
</header>
```

Understanding the example

Read the code from top to bottom. Notice how each opening tag identifies a piece of content, how attributes add information, and how nested elements establish relationships. The browser interprets this markup as a structured document rather than simply displaying the tag characters.

When applying this pattern in a real website, replace the sample words and file names with your own content. Save the document as an HTML file and open it in a modern browser to inspect the result. For images, store the image file separately so it can be replaced without rewriting the lesson.

Practical design note

Use the most meaningful HTML element for the job. Avoid selecting tags only because of their default appearance. Add CSS for layout and color, and check that the content remains understandable when styling is disabled. This separation makes a project easier to revise and maintain.

4.3 Main and Section Elements

The main element contains the primary content of the page and should generally appear once as the main landmark. Section groups thematically related content, typically with a heading. Not every container needs to be a section.

HTML example

```
<main>
  <h1>Learning Resources</h1>
  <section aria-labelledby="web-title">
    <h2 id="web-title">Web Development</h2>
    <p>Explore HTML and CSS.</p>
  </section>
</main>
```

Understanding the example

Read the code from top to bottom. Notice how each opening tag identifies a piece of content, how attributes add information, and how nested elements establish relationships. The browser interprets this markup as a structured document rather than simply displaying the tag characters.

When applying this pattern in a real website, replace the sample words and file names with your own content. Save the document as an HTML file and open it in a modern browser to inspect the result. For images, store the image file separately so it can be replaced without rewriting the lesson.

Practical design note

Use the most meaningful HTML element for the job. Avoid selecting tags only because of their default appearance. Add CSS for layout and color, and check that the content remains understandable when styling is disabled. This separation makes a project easier to revise and maintain.

4.4 Article and Aside Elements

An article is a self-contained composition, such as a news story or independent blog post. An aside holds complementary information related to surrounding content. The distinction helps users and assistive technologies understand the page.

HTML example

```
<article>
  <h2>How HTML Works</h2>
  <p>HTML structures web content.</p>
</article>
<aside><h2>Related Reading</h2></aside>
```

Understanding the example

Read the code from top to bottom. Notice how each opening tag identifies a piece of content, how attributes add information, and how nested elements establish relationships. The browser interprets this markup as a structured document rather than simply displaying the tag characters.

When applying this pattern in a real website, replace the sample words and file names with your own content. Save the document as an HTML file and open it in a modern browser to inspect the result. For images, store the image file separately so it can be replaced without rewriting the lesson.

Practical design note

Use the most meaningful HTML element for the job. Avoid selecting tags only because of their default appearance. Add CSS for layout and color, and check that the content remains understandable when styling is disabled. This separation makes a project easier to revise and maintain.

4.5 Footer, Figure, and Figcaption

A footer contains information about its nearest section or page, such as contact or copyright details. Figure groups self-contained media or examples. Figcaption supplies a visible explanation of the figure. These elements make educational pages easier to follow.

HTML example

```
<figure>
  <img src="semantic-layout.png" alt="Diagram of page landmarks">
  <figcaption>Figure 4.1: Semantic page layout.</figcaption>
</figure>
<footer><p>Contact the library team</p></footer>
```

Understanding the example

Read the code from top to bottom. Notice how each opening tag identifies a piece of content, how attributes add information, and how nested elements establish relationships. The browser interprets this markup as a structured document rather than simply displaying the tag characters.

When applying this pattern in a real website, replace the sample words and file names with your own content. Save the document as an HTML file and open it in a modern browser to inspect the result. For images, store the image file separately so it can be replaced without rewriting the lesson.

Practical design note

Use the most meaningful HTML element for the job. Avoid selecting tags only because of their default appearance. Add CSS for layout and color, and check that the content remains understandable when styling is disabled. This separation makes a project easier to revise and maintain.

Chapter 4 takeaway: The concepts in this chapter provide building blocks for readable, maintainable and accessible web documents.

Chapter 5: Creating Webpage Layouts

5.1 Organizing Page Content

Begin with an outline of what readers need. Put the site identity and navigation near the top, primary information in main, and related material in appropriate sections. HTML supplies the outline while CSS controls columns, spacing, and responsive design.

HTML example

```
<body>
  <header>Site header</header>
  <nav aria-label="Primary">Menu</nav>
  <main>Page content</main>
  <footer>Site information</footer>
</body>
```

Understanding the example

Read the code from top to bottom. Notice how each opening tag identifies a piece of content, how attributes add information, and how nested elements establish relationships. The browser interprets this markup as a structured document rather than simply displaying the tag characters.

When applying this pattern in a real website, replace the sample words and file names with your own content. Save the document as an HTML file and open it in a modern browser to inspect the result. For images, store the image file separately so it can be replaced without rewriting the lesson.

Practical design note

Use the most meaningful HTML element for the job. Avoid selecting tags only because of their default appearance. Add CSS for layout and color, and check that the content remains understandable when styling is disabled. This separation makes a project easier to revise and maintain.

5.2 Building a Semantic Webpage

A semantic page should have meaningful headings, a clear primary landmark, and logically grouped content. Avoid creating sections only to add borders or colors. Use CSS classes for visual styling rather than choosing an incorrect HTML element.

HTML example

```
<main>
  <h1>Course Catalog</h1>
  <section>
    <h2>First Year</h2>
    <article><h3>HTML Basics</h3></article>
  </section>
</main>
```

Understanding the example

Read the code from top to bottom. Notice how each opening tag identifies a piece of content, how attributes add information, and how nested elements establish relationships. The browser interprets this markup as a structured document rather than simply displaying the tag characters.

When applying this pattern in a real website, replace the sample words and file names with your own content. Save the document as an HTML file and open it in a modern browser to inspect the result. For images, store the image file separately so it can be replaced without rewriting the lesson.

Practical design note

Use the most meaningful HTML element for the job. Avoid selecting tags only because of their default appearance. Add CSS for layout and color, and check that the content remains understandable when styling is disabled. This separation makes a project easier to revise and maintain.

5.3 Forms and Input Elements

Forms collect user input. Each input should have a programmatically associated label. The name attribute identifies data submitted to a server. Choose input types such as email or number to provide appropriate browser behavior and validation.

HTML example

```
<form action="/subscribe" method="post">
  <label for="email">Email address</label>
  <input id="email" name="email" type="email" required>
  <button type="submit">Subscribe</button>
</form>
```

Understanding the example

Read the code from top to bottom. Notice how each opening tag identifies a piece of content, how attributes add information, and how nested elements establish relationships. The browser interprets this markup as a structured document rather than simply displaying the tag characters.

When applying this pattern in a real website, replace the sample words and file names with your own content. Save the document as an HTML file and open it in a modern browser to inspect the result. For images, store the image file separately so it can be replaced without rewriting the lesson.

Practical design note

Use the most meaningful HTML element for the job. Avoid selecting tags only because of their default appearance. Add CSS for layout and color, and check that the content remains understandable when styling is disabled. This separation makes a project easier to revise and maintain.

5.4 Accessibility and Best Practices

Accessible pages use descriptive headings, meaningful links, labels for controls, sufficient color contrast, keyboard-friendly interaction, and useful alternative text. Semantic HTML provides a strong starting point, but accessibility also depends on design and behavior.

HTML example

```
<label for="search">Search books</label>
<input id="search" type="search" name="q">
<button type="submit">Search</button>
```

Understanding the example

Read the code from top to bottom. Notice how each opening tag identifies a piece of content, how attributes add information, and how nested elements establish relationships. The browser interprets this markup as a structured document rather than simply displaying the tag characters.

When applying this pattern in a real website, replace the sample words and file names with your own content. Save the document as an HTML file and open it in a modern browser to inspect the result. For images, store the image file separately so it can be replaced without rewriting the lesson.

Practical design note

Use the most meaningful HTML element for the job. Avoid selecting tags only because of their default appearance. Add CSS for layout and color, and check that the content remains understandable when styling is disabled. This separation makes a project easier to revise and maintain.

5.5 Common HTML Mistakes

Frequent problems include missing alt text, skipped heading levels, generic links, invalid nesting, duplicate IDs, and using tables for page layout. Browser developer tools and HTML validators can help detect errors before publication.

HTML example

```
<!-- Avoid -->
<a href="guide.html">Click here</a>
<!-- Prefer -->
<a href="guide.html">Read the HTML guide</a>
```

Understanding the example

Read the code from top to bottom. Notice how each opening tag identifies a piece of content, how attributes add information, and how nested elements establish relationships. The browser interprets this markup as a structured document rather than simply displaying the tag characters.

When applying this pattern in a real website, replace the sample words and file names with your own content. Save the document as an HTML file and open it in a modern browser to inspect the result. For images, store the image file separately so it can be replaced without rewriting the lesson.

Practical design note

Use the most meaningful HTML element for the job. Avoid selecting tags only because of their default appearance. Add CSS for layout and color, and check that the content remains understandable when styling is disabled. This separation makes a project easier to revise and maintain.

Chapter 5 takeaway: The concepts in this chapter provide building blocks for readable, maintainable and accessible web documents.

Chapter 6: Practical HTML Examples

6.1 Creating a Basic Webpage

Start with a valid document and a clear title. Add one main heading and a few paragraphs. Save the file with an .html extension, then open it in a browser. Inspect how the browser displays each element.

HTML example

```
<!doctype html>
<html lang="en">
<head><meta charset="utf-8"><title>My Page</title></head>
<body><main><h1>Hello, Web!</h1><p>My first page.</p></main></body>
</html>
```

Understanding the example

Read the code from top to bottom. Notice how each opening tag identifies a piece of content, how attributes add information, and how nested elements establish relationships. The browser interprets this markup as a structured document rather than simply displaying the tag characters.

When applying this pattern in a real website, replace the sample words and file names with your own content. Save the document as an HTML file and open it in a modern browser to inspect the result. For images, store the image file separately so it can be replaced without rewriting the lesson.

Practical design note

Use the most meaningful HTML element for the job. Avoid selecting tags only because of their default appearance. Add CSS for layout and color, and check that the content remains understandable when styling is disabled. This separation makes a project easier to revise and maintain.

6.2 Building a Student Profile Page

A student profile can use a main heading, a figure for a profile image, and sections for academic details and interests. Use neutral, meaningful markup so the content remains understandable without visual styling.

HTML example

```
<main>
  <h1>Student Profile</h1>
  <figure><img src="student.jpg" alt="Student portrait"></figure>
  <section><h2>About Me</h2><p>BSIT student.</p></section>
</main>
```

Understanding the example

Read the code from top to bottom. Notice how each opening tag identifies a piece of content, how attributes add information, and how nested elements establish relationships. The browser interprets this markup as a structured document rather than simply displaying the tag characters.

When applying this pattern in a real website, replace the sample words and file names with your own content. Save the document as an HTML file and open it in a modern browser to inspect the result. For images, store the image file separately so it can be replaced without rewriting the lesson.

Practical design note

Use the most meaningful HTML element for the job. Avoid selecting tags only because of their default appearance. Add CSS for layout and color, and check that the content remains understandable when styling is disabled. This separation makes a project easier to revise and maintain.

6.3 Creating a Navigation Menu

A navigation menu should group important links and describe where each one leads. A list inside nav is a common pattern. Keep the order consistent across pages and ensure links can be reached using a keyboard.

HTML example

```
<nav aria-label="Main menu">
  <ul>
    <li><a href="index.html">Home</a></li>
    <li><a href="books.html">Books</a></li>
    <li><a href="about.html">About</a></li>
  </ul>
</nav>
```

Understanding the example

Read the code from top to bottom. Notice how each opening tag identifies a piece of content, how attributes add information, and how nested elements establish relationships. The browser interprets this markup as a structured document rather than simply displaying the tag characters.

When applying this pattern in a real website, replace the sample words and file names with your own content. Save the document as an HTML file and open it in a modern browser to inspect the result. For images, store the image file separately so it can be replaced without rewriting the lesson.

Practical design note

Use the most meaningful HTML element for the job. Avoid selecting tags only because of their default appearance. Add CSS for layout and color, and check that the content remains understandable when styling is disabled. This separation makes a project easier to revise and maintain.

6.4 Designing an Educational Website

An educational site benefits from clear course categories, descriptive book titles, accessible image alternatives, and organized lesson pages. The document structure should remain meaningful even if CSS fails to load.

HTML example

```
<main>
  <h1>BSIT E-Library</h1>
  <section><h2>First Year Books</h2>
    <article><h3>HTML Fundamentals</h3>
      <a href="html-book.html">Read book</a>
    </article>
  </section>
</main>
```

Understanding the example

Read the code from top to bottom. Notice how each opening tag identifies a piece of content, how attributes add information, and how nested elements establish relationships. The browser interprets this markup as a structured document rather than simply displaying the tag characters.

When applying this pattern in a real website, replace the sample words and file names with your own content. Save the document as an HTML file and open it in a modern browser to inspect the result. For images, store the image file separately so it can be replaced without rewriting the lesson.

Practical design note

Use the most meaningful HTML element for the job. Avoid selecting tags only because of their default appearance. Add CSS for layout and color, and check that the content remains understandable when styling is disabled. This separation makes a project easier to revise and maintain.

6.5 Complete Semantic HTML Page

A complete semantic document combines metadata, navigation, content landmarks, structured sections, and a footer. Use it as a reusable starting point, then adjust the content and CSS to fit a specific project.

HTML example

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Learning Portal</title>
</head>
<body>
<header><h1>Learning Portal</h1></header>
<nav aria-label="Primary"><a href="index.html">Home</a></nav>
<main><article><h2>HTML Lesson</h2><p>Welcome!</p></article></main>
<footer><p>Learning Portal</p></footer>
</body>
</html>
```

Understanding the example

Read the code from top to bottom. Notice how each opening tag identifies a piece of content, how attributes add information, and how nested elements establish relationships. The browser interprets this markup as a structured document rather than simply displaying the tag characters.

When applying this pattern in a real website, replace the sample words and file names with your own content. Save the document as an HTML file and open it in a modern browser to inspect the result. For images, store the image file separately so it can be replaced without rewriting the lesson.

Practical design note

Use the most meaningful HTML element for the job. Avoid selecting tags only because of their default appearance. Add CSS for layout and color, and check that the content remains understandable when styling is disabled. This separation makes a project easier to revise and maintain.

Chapter 6 takeaway: The concepts in this chapter provide building blocks for readable, maintainable and accessible web documents.